

KPIT APEX LAB INTERNSHIP

Project Documentation

Face Tracking and Detection Robot

Developed on Firebird V (ATmega2560) + Raspberry Pi 5

*MATLAB/Simulink Exploration → Final Python Implementation
(YOLOv8n, Voice Control, Obstacle Avoidance)*

Internship Period: 11 May 2026 – 3 July 2026

KPIT Apex Lab

Authors

Darpan Dongre
Pratham Modi
Amay Bembde

Contents

1. Abstract	4
1 Introduction	5
1.1 Background	5
1.2 Problem Statement	5
1.3 Objectives	5
1.4 Constraints	5
1.5 Pre-Project Learning	6
2 System Overview	7
2.1 Hardware Components	7
2.2 Software and Toolboxes (MATLAB Phase)	7
2.3 High-Level System Architecture	8
3 Detection Approaches: Detailed Technical Analysis	9
3.1 Tiny YOLOv4 — The Initial Working Solution	9
3.1.1 What is YOLO?	9
3.1.2 Core Concept: Grid-Based Detection	9
3.1.3 Tiny YOLOv4 Architecture	9
3.1.4 Why YOLOv4 Was Selected Initially	10
3.2 YOLOv8n — Attempted Migration	10
3.2.1 What is YOLOv8?	10
3.2.2 YOLOv8n Architecture	10
3.2.3 MATLAB YOLOv8n Integration Attempt	11
3.2.4 Outcome: Same Camera Frame Lag	11
3.3 Haar Cascade — Real-Time Camera but No Detection	11
3.3.1 What are Haar Cascades?	11
3.3.2 MATLAB Implementation Attempt	11
3.3.3 Outcome: Camera Real-Time but No Detections	12
3.4 MobileNet-SSD — Discovered to be Classifier Only	12
3.4.1 What is MobileNet-SSD?	12
3.4.2 Outcome: Classifier Without Bounding Boxes	12
3.5 HOG + SVM Pedestrian Detector	13
3.5.1 What is HOG Detection?	13
3.5.2 Outcome: Detecting Anything Moving	13
3.6 ACF (Aggregated Channel Features) Face Detector — Real-Time Camera but Poor Tracking	13
3.6.1 What is ACF Detection?	13
3.6.2 Deployment on Raspberry Pi	14
3.6.3 Problems Encountered	14
3.6.4 Outcome: Proof of Concept, Not a Final Solution	15
3.7 YOLOX — Suggested by MathWorks Support	15

3.7.1	What is YOLOX?	15
3.7.2	Outcome: Same Camera Frame Lag	15
3.8	ROS (Robot Operating System) — Studied but Not Adopted	16
3.8.1	What is ROS?	16
3.8.2	Why ROS Was Not Adopted	16
4	Iteration History and Problem-Solving Timeline	17
5	Phase 1: MATLAB/Simulink Implementation (Working System)	18
5.1	Simulink Model Structure	18
5.2	Person Detection Pipeline	18
5.3	Frame-Skip Optimisation	18
5.4	Tracking and Lock Logic (lockAndDecide)	19
5.5	Motor Command Decision	19
5.6	Communication Protocol	19
5.7	Watchdog Safety Mechanism	20
5.8	MATLAB Source Code	20
5.8.1	detectPersons — Top-Level Entry Point	20
5.8.2	yoloDetect — Model Inference	20
5.8.3	ATmega2560 UART2 ISR	21
6	Phase 1 Testing and Results	22
6.1	Test Cases	22
6.2	Performance Observations	22
7	Challenges and Solutions (Complete)	24
8	Key Technical Learnings	26
9	Phase 2: Python Implementation (Completed)	27
9.1	Overview and Final Architecture	27
9.2	Detection Pipeline	27
9.3	Threaded Camera Capture	28
9.4	Target Lock and Proportional Steering	28
9.4.1	FaceLock: Centre-Point Target Persistence	28
9.4.2	Proportional Steering: decide_command()	29
9.4.3	Momentum on Brief Loss and Serial Debounce	29
9.5	Voice Command Subsystem (Start / Stop Control)	29
9.6	Live Video Streaming Dashboard	30
10	Obstacle Avoidance Subsystem (ATmega2560 Firmware)	31
10.1	Motivation and Control Philosophy	31
10.2	Sensors Used	31
10.3	Avoidance State Machine	32
10.4	Anti-Spin Turn-Burst Logic (Tracking Turns)	33
10.5	LCD Status Display	33
10.6	Watchdog Safety (Retained from Phase 1)	33
11	Phase 2 Testing and Results	34
11.1	Test Cases	34
11.2	Performance Observations	35
12	Challenges and Solutions (Phase 2)	36

13 Future Scope	38
13.1 Planned Hardware Improvements	38
13.2 Planned Software Improvements	38
14 Conclusion	39
References	40

1. Abstract

This document describes the complete design, development, iterative problem-solving, and final implementation of a real-time Face Tracking and Detection Robot built during a 8 weeks internship at KPIT Apex Lab (May–July 2026). The system integrates a Raspberry Pi 5 with a Firebird V wheeled robot platform (ATmega2560 microcontroller) to achieve autonomous, voice-controllable, obstacle-aware person/face-following behaviour.

The project went through multiple significant architectural iterations. The initial Simulink pipeline used Tiny YOLOv4 and achieved functional person tracking but suffered from approximately one second of end-to-end latency. Subsequent attempts to further reduce latency or change the detection approach explored YOLOv8n (official MATLAB GitHub), Haar cascade face detection, MobileNet-SSD, HOG-based pedestrian detection, YOLOX, and a study of ROS — all within the MATLAB/Simulink constraint. Each approach revealed specific limitations documented in this report. The project was ultimately completed as a pure Python implementation running directly on the Raspberry Pi 5, which removed the MATLAB/Simulink constraint and enabled access to the full PyTorch/Ultralytics ecosystem. The final system adds two further capabilities beyond basic tracking: an offline, voice-controlled start/stop subsystem, and an autonomous obstacle-avoidance manoeuvre implemented directly on the ATmega2560 firmware.

Chapter 1

Introduction

1.1 Background

Mobile robots capable of perceiving and following humans have broad applications in surveillance, assistive robotics, logistics, and human-robot interaction. Deploying such capabilities on resource-constrained embedded hardware — rather than a desktop workstation with a discrete GPU — presents meaningful engineering challenges around inference latency, power budgets, and software architecture. This project addresses those challenges starting within the KPIT Apex Lab toolchain (MATLAB and Simulink) and later extending to Python-based solutions after exhausting the MATLAB/Simulink approach.

1.2 Problem Statement

Given a wheeled robot (Firebird V) and a single-board computer (Raspberry Pi 5), design and implement a closed-loop system that: (a) captures live video from a camera, (b) detects a person or face in the frame using a pre-trained model, (c) computes a directional command to keep the detected target centred, and (d) drives the robot's motors accordingly — all in real time.

1.3 Objectives

- Deploy a real-time person/face detection pipeline on Raspberry Pi 5.
- Use pre-trained models without training from scratch.
- Implement target lock logic robust to brief detection misses and frame jitter.
- Transmit directional motor commands from the Pi to the Firebird V over UART serial.
- Implement a hardware watchdog on the ATmega2560 for fail-safe motor stop.
- Achieve a demonstrable prototype within the 1.5-month internship window.

1.4 Constraints

- Phase 1 (completed): Entire pipeline must use MATLAB and Simulink only — no Python.
- Phase 2 (current): Python-only implementation on Raspberry Pi 5, no MATLAB dependency.
- Hardware fixed: Firebird V robot, Raspberry Pi 5, USB webcam / Pi Camera Module.

- No GPU: all inference must run on the Pi 5's ARM Cortex-A76 CPU cores.
- Timeline: approximately 6–8 weeks (11 May – early July 2026).

1.5 Pre-Project Learning

Before development began, the team completed the following MathWorks MATLAB Onramp courses assigned by the project mentor:

- MATLAB Onramp — Machine Learning
- MATLAB Onramp — Computer Vision
- MATLAB Onramp — Deep Learning

These provided the foundational understanding of MATLAB's Computer Vision Toolbox, Deep Learning Toolbox, and model deployment workflow. Additionally, background reading on YOLO architectures, IoU-based tracking, serial communication protocols, and embedded watchdog design was conducted before and during development.

Chapter 2

System Overview

2.1 Hardware Components

Component	Specification / Role
Firebird V Robot	ATmega2560-based wheeled robot platform (eYantra). Receives serial commands and drives DC motors via H-bridge on PORTA.
Raspberry Pi 5	Main compute board (ARM Cortex-A76, 4-core, 64-bit). Runs inference pipeline. Connected to webcam via USB and to Firebird V via USB-UART.
USB Webcam	Captures live video at 160x120 (MATLAB phase) or 640x480 (Python phase) resolution via USB.
USB Cable (Pi to Robot)	Provides the serial data link (UART2 on ATmega2560 via onboard USB-UART bridge) and optionally power during development.
Host PC (MATLAB Phase)	MATLAB R2026a used for model development, Simulink model building, code generation, and deployment over Wi-Fi.

2.2 Software and Toolboxes (MATLAB Phase)

Toolbox / Package	Usage
MATLAB R2026a	Primary development environment. All algorithm code written in MATLAB.
Simulink R2026a	Block diagram model: V4L2 Source → MATLAB Function → Serial Send.
Computer Vision Toolbox	Image processing utilities, bounding box operations, Haar cascade detector.
Deep Learning Toolbox	Hosts Tiny YOLOv4 COCO / YOLOv8n models; runs inference via <code>detect()</code> .
Raspberry Pi Blockset	V4L2 Source block (webcam capture), Serial Send block, Pi deployment support.
Simulink Coder + Embedded Coder	Generates and optimises C code for ARM Cortex-A76 on Pi 5.
MATLAB Coder	Compiles MATLAB functions (<code>detectPersons</code> , <code>lockAndDecide</code> , <code>yoloDetect</code>) to C for codegen deployment.

Toolbox / Package	Usage
Atmel Studio 7	ATmega2560 firmware development, compilation, and flashing.

2.3 High-Level System Architecture

The system operates as a closed-loop pipeline across two processing nodes:

Raspberry Pi 5 (Inference Node):

```

[Camera / Webcam]
  |
  v
V4L2 Source / OpenCV VideoCapture (160x120 or 640x480, RGB)
  |
  v
Detection Model (YOLOv4 / YOLOv8n / Haar / HOG / Python YOLO)
  |
  v
lockAndDecide() -> 'F' / 'L' / 'R' / 'S'
  |
  v
Serial Send (115200 baud, /dev/ttyUSB0)

```

Firebird V (ATmega2560):

```

UART2 RX ISR <- receives single ASCII byte
  |
  v
Switch 'F'/'L'/'R'/'S' -> motor primitives
  |
  v
PORTA -> H-bridge -> DC motors
  |
  Watchdog Timer (1s): wdt_reset() on each byte
  L- Pi freeze -> no bytes -> WDT fires -> motor_stop()

```

Chapter 3

Detection Approaches: Detailed Technical Analysis

3.1 Tiny YOLOv4 — The Initial Working Solution

3.1.1 What is YOLO?

YOLO (You Only Look Once) is a family of real-time object detection neural networks introduced by Joseph Redmon and Ali Farhadi in 2016. Unlike two-stage detectors such as Faster R-CNN (which first proposes candidate regions then classifies them), YOLO frames detection as a single regression problem: it takes an image as input and simultaneously outputs bounding box coordinates and class probabilities for all objects in a single forward pass through the network. This unified architecture is the fundamental reason YOLO is fast enough for real-time applications.

3.1.2 Core Concept: Grid-Based Detection

YOLO divides the input image into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose centre falls within that cell. Each cell predicts B bounding boxes and a confidence score for each box. The confidence score reflects both how likely the box contains an object and how accurately the box is predicted (measured as Intersection over Union between predicted and ground-truth box). Each cell also predicts C class probability scores. The final output tensor has shape $S \times S \times (B \times 5 + C)$, where 5 accounts for x, y, w, h coordinates and objectness confidence.

3.1.3 Tiny YOLOv4 Architecture

Tiny YOLOv4 is a lightweight variant of YOLOv4 designed for deployment on resource-constrained devices. Its architecture consists of:

- **Backbone:** CSPDarknet53-tiny — a stripped-down version of the CSPDarknet53 backbone used in full YOLOv4. Uses Cross Stage Partial (CSP) connections to reduce computation while maintaining representational capacity.
- **Neck:** SPP (Spatial Pyramid Pooling) module with three pooling windows (5x5, 9x9, 13x13) concatenated to capture multi-scale context. PANet (Path Aggregation Network) shortened to two feature pyramid levels (P4 and P5) for feature fusion.
- **Head:** Two detection heads at two scales — one for larger objects (detected at coarser feature map) and one for smaller objects (at finer feature map). Each head predicts 3 anchor boxes per grid cell.

- **Activation:** Mish activation in the backbone (a smooth, non-monotonic function outperforming ReLU in accuracy) and Leaky ReLU in the detection head.
- **Input:** 416x416 (standard) or 320x320 as used in this project for speed.
- **COCO Classes:** 80 classes including ‘person’, which is the class used for tracking.

In MATLAB, Tiny YOLOv4 is available as `tinyyolov4coco.mat` via the Deep Learning Toolbox. Inference is invoked through `detect(yolov4Obj, frame, 'Threshold', 0.4)` which runs a full forward pass and applies non-maximum suppression (NMS) to eliminate overlapping detections.

3.1.4 Why YOLOv4 Was Selected Initially

- Available as a ready-to-use `.mat` file — no custom training pipeline required.
- Detects the ‘person’ class which is more robust than face-only detection (works when occluded or at distance).
- Acceptable inference speed on Pi 5 ARM Cortex-A76 (~700–800ms per inference call at 320x320).
- Well-integrated into MATLAB’s codegen workflow via `coder.loadDeepLearningNetwork()`.

3.2 YOLOv8n — Attempted Migration

3.2.1 What is YOLOv8?

YOLOv8 is the eighth major generation of the YOLO series, released by Ultralytics in January 2023. It represents a significant architectural modernisation over YOLOv4/v5, introducing an anchor-free detection head (eliminating the need to predefine anchor box shapes), a new C2f (Cross Stage Partial with 2 bottleneck blocks) module in the backbone replacing C3, and a decoupled head that separates objectness and classification predictions. YOLOv8 achieves substantially better accuracy-speed tradeoffs than its predecessors on the COCO benchmark.

3.2.2 YOLOv8n Architecture

YOLOv8n (nano) is the smallest variant in the YOLOv8 family, optimised for edge inference:

- **Backbone:** CSPDarknet with C2f modules. The C2f module replaces the C3 module of YOLOv5, adding more gradient flow paths for better feature reuse with fewer parameters.
- **Neck:** PAN-FPN (Path Aggregation Network with Feature Pyramid Network) for multi-scale feature fusion across three scales: P3 (small objects), P4 (medium), P5 (large).
- **Head:** Anchor-free decoupled head — one branch for bounding box regression (using Distribution Focal Loss), another for classification. This removes the need to tune anchor hyperparameters.
- **Parameters:** ~3.2M (versus ~5.8M for YOLOv4-tiny). Significantly more efficient for edge deployment.
- **Input:** 640x640 default, but can be resized. The model’s anchor-free design means it handles arbitrary resolutions more gracefully.
- **COCO mAP:** YOLOv8n achieves ~37.3 mAP@50:95, substantially higher than Tiny YOLOv4’s ~21.7 mAP.

3.2.3 MATLAB YOLOv8n Integration Attempt

The official MathWorks GitHub repository provides a YOLOv8n model converted to a MATLAB-compatible format. The attempt to integrate this into the Simulink pipeline mirrored the YOLOv4 approach:

- Model loaded via `coder.loadDeepLearningNetwork()` with the converted YOLOv8n .mat file.
- `detect()` called with the 320x320 resized frame, filtering for 'person' class.
- Same `lockAndDecide()` tracking logic applied downstream.

3.2.4 Outcome: Same Camera Frame Lag

The YOLOv8n model did not resolve the camera frame lag issue. Despite YOLOv8n having fewer parameters than Tiny YOLOv4, the inference time within the Simulink synchronous execution context remained in the same 600–900ms range on the Pi 5 ARM cores without NEON SIMD optimisation. The Simulink model step blocked on `detect()` just as it did with YOLOv4, resulting in identical latency characteristics. This confirmed that the latency bottleneck was not model-specific but architectural — the synchronous Simulink execution model on an ARM CPU without hardware acceleration was the root constraint.

3.3 Haar Cascade — Real-Time Camera but No Detection

3.3.1 What are Haar Cascades?

Haar cascades are a classical computer vision technique for object detection introduced by Paul Viola and Michael Jones in their 2001 paper 'Rapid Object Detection using a Boosted Cascade of Simple Features'. Unlike deep learning detectors, Haar cascades use handcrafted features rather than learned representations.

The algorithm works in three stages. First, Haar features are computed: rectangular filters (similar to Haar wavelets) applied to integral image representations of the input. These features capture edge, line, and diagonal gradient patterns at multiple scales. Second, a cascade of boosted classifiers is built using AdaBoost. Each stage in the cascade is a strong classifier composed of many weak classifiers (each a single Haar feature threshold). The cascade is structured so early stages rapidly reject non-face regions using simple features, and only candidate regions that pass all stages are classified as faces. Third, a sliding window scans the image at multiple scales (image pyramid), applying the cascade at each position and scale.

The key advantage is speed: on a CPU, Haar cascade face detection runs in real time even on single-board computers. The key disadvantage is sensitivity to lighting, pose, and occlusion — it was trained primarily on frontal, well-lit faces.

3.3.2 MATLAB Implementation Attempt

The `vision.CascadeObjectDetector` system object in MATLAB's Computer Vision Toolbox provides a pre-trained frontal face Haar cascade. The integration was straightforward — replace the YOLO block with:

```
detector = vision.CascadeObjectDetector('FrontalFaceCART');  
bboxes = step(detector, frame);
```

This removed the deep learning inference overhead entirely, allowing the Simulink pipeline to run at near-camera frame rate. The camera feed lag dropped dramatically — the V4L2 Source block delivered frames in real time since `detect()` was no longer blocking.

3.3.3 Outcome: Camera Real-Time but No Detections

While the camera feed ran in real time, the Haar cascade consistently failed to produce any bounding box detections. The root causes were:

- **Low resolution:** the 160x120 frame (even resized to 320x320) provided insufficient detail for the cascade classifier, which was trained on higher-resolution face crops.
- **Viewing angle:** the robot camera captured faces at varying angles and distances. Haar cascade frontal face detection requires near-frontal, upright face poses. Profile or tilted faces are not detected by the standard model.
- **Lighting conditions:** the lab lighting introduced shadows on faces that degraded cascade confidence below the detection threshold.
- **No person-class option:** unlike YOLO which detects the broader ‘person’ class (body + face), the cascade is strictly face-specific, making it more sensitive to visibility of the face region.

Alternative cascades (profile face, upper body) were briefly explored but similarly produced no reliable detections in the test environment.

3.4 MobileNet-SSD — Discovered to be Classifier Only

3.4.1 What is MobileNet-SSD?

MobileNet-SSD is a lightweight object detection architecture combining the MobileNetV1 backbone with the Single Shot Multibox Detector (SSD) detection head. MobileNetV1 introduced depthwise separable convolutions: factoring a standard 3D convolution into a depthwise convolution (per-channel spatial filtering) followed by a pointwise 1x1 convolution (cross-channel mixing). This reduces computation by a factor of 8–9x compared to standard convolutions with minimal accuracy loss.

SSD (Single Shot Multibox Detector) detects objects at multiple scales by attaching detection heads to multiple layers in the backbone’s feature hierarchy. This multi-scale approach allows detecting both small and large objects without a separate region proposal stage.

MobileNet-SSD trained on COCO or VOC can detect 20–80 classes including ‘person’, making it theoretically suitable for this project.

3.4.2 Outcome: Classifier Without Bounding Boxes

Investigation revealed that the version of MobileNet available in MATLAB’s Deep Learning Toolbox at the time of testing was the classification-only `mobilenetv2` model (or similar), which outputs class label probabilities for the entire image but does not produce bounding box coordinates or support the `detect()` API. This is a fundamentally different task — image classification versus object detection. A classifier can answer ‘is there a person in this image?’ but cannot answer ‘where in the image is the person?’ — the spatial coordinate output required for robot steering. As a result, MobileNet-SSD was abandoned as it could not fulfil the core requirement of bounding box output without a separate detection wrapper that was not available in the installed toolbox version.

3.5 HOG + SVM Pedestrian Detector

3.5.1 What is HOG Detection?

Histogram of Oriented Gradients (HOG) is a classical feature descriptor for object detection introduced by Navneet Dalal and Bill Triggs in 2005. HOG computes gradient magnitudes and orientations across cells of an image, then aggregates them into normalised histograms. These histograms capture local edge and shape information. A linear SVM (Support Vector Machine) classifier trained on these features can detect pedestrians with high accuracy on the original DPM datasets.

In MATLAB, `people.detectPeopleACF()` or the HOG-based `peopleDetector` (via `vision.PeopleDetector`) provides a pre-trained pedestrian detector. The detector slides a window across the image, computes HOG features, and returns a confidence score. This approach requires no GPU and runs efficiently on CPU.

3.5.2 Outcome: Detecting Anything Moving

The HOG-based pedestrian detector proved highly unreliable in the lab environment. The core problems were:

- **High false positive rate:** the detector flagged chairs, table legs, door frames, and moving objects (bag, laptop, coat on a stand) as pedestrians. The lab's cluttered background with many vertical structures closely matched the HOG feature patterns of human legs and torsos.
- **Scale sensitivity:** the detector is calibrated for pedestrians at a specific scale range. A person sitting, crouching, or at close range fell outside the detector's optimal scale window.
- **Motion response:** any moving object in the frame that produced gradient changes similar to a walking person triggered spurious detections, making the robot turn erratically toward furniture and equipment.
- **No lock robustness:** unlike YOLO's bbox output which can be IoU-matched across frames, HOG returns overlapping boxes at multiple scales requiring careful NMS tuning that was not achievable within the Simulink codegen constraints.

HOG detection was abandoned due to its unacceptably high false positive rate making robot behaviour unsafe.

3.6 ACF (Aggregated Channel Features) Face Detector — Real-Time Camera but Poor Tracking

3.6.1 What is ACF Detection?

Aggregated Channel Features (ACF) is a classical object detection framework introduced by Piotr Dollar et al. ACF detection works by extracting multiple feature channels from an input image — colour information (LUV colour space), gradient magnitude, and gradient orientation histograms — then aggregating and downsampling these channels. A boosted cascade classifier (similar in spirit to Viola-Jones but using richer features) is trained on these aggregated channels to determine whether a sliding window region contains the target object (in this case, a face).

The detection process per frame is:

1. Capture RGB image from the camera.

2. Compute feature channels: LUV colour channels, gradient magnitude, 6-orientation gradient histogram channels.
3. Aggregate and downsample channel information across a spatial neighbourhood.
4. Slide the trained cascade classifier across the image at multiple scales (image pyramid).
5. Apply non-maximum suppression to merge overlapping detections.
6. Output face bounding boxes and confidence scores.

ACF detectors are significantly faster than deep learning detectors on CPU hardware and are fully compatible with MATLAB/Simulink code generation, making them attractive for embedded deployment. A custom ACF detector was trained by a groupmate using a pipeline built on:

- WIDER FACE dataset plus MATLAB sample videos (`visionface.avi`, `tilted_face.avi`, `vipmen.avi`).
- Preprocessing: removal of invalid annotations, extremely small faces, and bounding box errors.
- Augmentation: horizontal flipping, brightened and darkened variations, hard negative mining.
- Final model trained on approximately 3500 images across eight training stages.

To improve stability during deployment, the pipeline incorporated bounding box smoothing (EMA), IoU-based target locking, Kalman filter prediction for inter-frame association, and a majority-vote command buffer to suppress command jitter.

3.6.2 Deployment on Raspberry Pi

The ACF detector was deployed via a MATLAB Function block in Simulink following the same pipeline structure as the YOLOv4 implementation: Camera Frame → ACF Detector → Target Selection → Kalman Filter → Robot Command Generation. The detected face centre was compared with the frame centre to generate directional motor commands.

The critical advantage observed was that the ACF detector ran in real time — camera frames were delivered and processed at full frame rate without the blocking inference latency that plagued all deep learning approaches. The absence of a GPU-scale neural network forward pass meant the Simulink step completed quickly, keeping the camera feed live.

3.6.3 Problems Encountered

Despite the real-time camera performance, the ACF detector exhibited several tracking reliability problems that made it unsuitable as a final solution:

- **Bounding box jitter:** the detector produced slightly different bounding boxes for the same face in consecutive frames, causing the tracking marker and robot commands to fluctuate even when the person was stationary. EMA smoothing and Kalman filtering partially mitigated this but did not eliminate it.
- **Loss of tracking on rapid movement:** fast head movement caused the newly detected face position to differ significantly from the Kalman-predicted position, breaking IoU-based association and dropping the lock.
- **Multiple spurious detections:** at lower detection thresholds, the detector occasionally returned multiple bounding boxes for a single face, causing unstable target selection.

- **Environmental sensitivity:** detection accuracy degraded under partial occlusion, large face rotations, illumination changes, and long-range detection — all conditions common in the lab environment.
- **Training memory constraints:** increasing training stages and negative samples to improve robustness caused out-of-memory errors during training, requiring reductions in training complexity and dataset size, which in turn limited detector accuracy.
- **Unreliable direction classification:** despite the majority-vote command buffer and widened dead zone, the direction commands (Left/Right/Forward) remained jittery due to the underlying bounding box instability, making the robot's physical response erratic.

3.6.4 Outcome: Proof of Concept, Not a Final Solution

The ACF detector successfully demonstrated that a classical computer vision approach can deliver real-time camera throughput on the Raspberry Pi within the Simulink framework — something no deep learning model achieved. However, the tracking robustness was insufficient for reliable autonomous face following. The combination of jitter, tracking loss, and environmental sensitivity meant the robot's behaviour was unpredictable in normal lab conditions.

The ACF implementation is therefore classified as a successful proof of concept for real-time embedded detection but an unsuitable final solution. Its findings reinforced that the project requires a more robust detection backbone — motivating the transition to a Python-based implementation with full access to the Ultralytics ecosystem.

3.7 YOLOX — Suggested by MathWorks Support

3.7.1 What is YOLOX?

YOLOX is an anchor-free evolution of YOLO released by Megvii in 2021. It introduced two major improvements over YOLOv3/v4: a decoupled head (separating classification and bounding box regression into parallel branches, similar to what YOLOv8 later adopted) and a SimOTA label assignment strategy (optimal transport assignment replacing manual anchor matching). These changes substantially improved accuracy, particularly for small objects and crowded scenes. YOLOX-Nano and YOLOX-Tiny variants are specifically designed for edge deployment.

MathWorks support recommended YOLOX as a potentially faster alternative to YOLOv4 within the MATLAB/Simulink workflow, as a pretrained MATLAB-compatible model was available.

3.7.2 Outcome: Same Camera Frame Lag

YOLOX showed identical latency characteristics to YOLOv4 and YOLOv8n within the Simulink synchronous execution model. Even though YOLOX-Tiny has fewer parameters than Tiny YOLOv4 on paper, the inference time on the Pi 5 ARM cores within MATLAB's generated C runtime remained in the 600–900ms range. This definitively confirmed the conclusion that the problem was not which deep learning model was used, but the fundamental mismatch between the Simulink synchronous stepping model and the variable-latency nature of neural network inference on an ARM CPU. No MATLAB/Simulink-based deep learning model was going to solve the latency problem without hardware acceleration (GPU, NPU) or asynchronous execution — neither of which was available or supported in the Simulink workflow for the Pi 5.

3.8 ROS (Robot Operating System) — Studied but Not Adopted

3.8.1 What is ROS?

ROS (Robot Operating System) is an open-source robotics middleware framework providing a publish-subscribe communication infrastructure, hardware abstraction, device drivers, libraries for common robotics functions, and tools for visualisation and simulation. ROS 2 (the current generation) runs on Linux and supports real-time communication via DDS (Data Distribution Service).

ROS uses a node-based architecture: each computational unit (camera driver, detection model, motor controller) is a separate node publishing and subscribing to named topics. This decoupled architecture naturally solves the synchronous blocking problem — a detection node can publish results asynchronously without blocking the camera or motor nodes.

3.8.2 Why ROS Was Not Adopted

After studying ROS architecture and its potential to solve the synchronous execution problem, the team concluded it was not the right choice for this project:

- **Complexity overhead:** ROS introduces significant setup complexity for a standalone two-node system (Pi + ATmega). The publisher-subscriber model, package management (`colcon build`), and DDS configuration represent substantial engineering overhead for what is fundamentally a simple serial communication requirement.
- **ATmega2560 integration:** the ATmega2560 is not a ROS-capable microcontroller. Integrating it into a ROS network requires micro-ROS, which demands additional middleware on the MCU side and was not supported in the current firmware environment (Atmel Studio 7 / bare-metal C).
- **Standalone robot constraint:** the project requirement is a fully standalone robot (no external PC during operation). ROS typically requires a host machine for the `roscore` master node (ROS 1) or careful DDS configuration (ROS 2). Running full ROS 2 on a Pi 5 while simultaneously performing inference would add OS-level overhead counterproductive to latency goals.
- **Python sufficiency:** the core asynchronous execution problem that motivated studying ROS is directly and more simply solved by Python's `threading` module (running inference in a background thread while the main loop handles serial communication). ROS would be valuable in a multi-robot or multi-sensor system; for this single-robot application, it adds complexity without commensurate benefit.

Chapter 4

Iteration History and Problem-Solving Timeline

Phase	Approach	Outcome	Root Cause
1	Tiny YOLOv4 in Simulink (320x320, every 3rd frame)	WORKING — $\sim 1s$ latency, person tracking functional	Baseline system achieved
2	YOLOv8n from MathWorks GitHub in Simulink	FAILED — same camera frame lag	Synchronous Simulink model blocks on ARM inference regardless of model size
3	Haar Cascade face detection (MATLAB <code>vision.CascadeObjectDetector</code>)	PARTIAL — camera real-time, zero detections	Low resolution + off-angle faces + cluttered background defeat cascade
4	MobileNet-SSD (MATLAB Deep Learning Toolbox)	FAILED — no bounding box output	Available model is classification-only, not a detector
5	HOG pedestrian detector (MATLAB <code>vision.PeopleDetector</code>)	FAILED — false detections everywhere	High false positive rate on lab furniture and moving objects
6	ACF face detector (custom trained, MATLAB Function block in Simulink)	PARTIAL — real-time camera, but poor tracking robustness	Bounding box jitter and tracking loss; insufficient for reliable autonomous following
7	YOLOX (recommended by MathWorks support)	FAILED — same camera frame lag	Confirms Simulink synchronous model is the architectural bottleneck, not the choice of model
8	ROS study	NOT ADOPTED — unnecessary complexity for standalone robot	Python threading solves the same problem more simply
9 (Current)	Python-only implementation (Ultralytics YOLOv8n + OpenCV)	IN PROGRESS	Direct process control, async threading, no Simulink overhead

Chapter 5

Phase 1: MATLAB/Simulink Implementation (Working System)

5.1 Simulink Model Structure

The Simulink model consists of three blocks connected in series:

Block	Configuration
V4L2 Video Source	Resolution: 160x120. Output: separate R, G, B ports. Device: /dev/video0.
MATLAB Function Block	Function: <code>detectPersons(R, G, B)</code> . Output: <code>serialCmd(uint8)</code> .
Serial Send Block	Port: /dev/ttyUSB0. Baud: 115200. Data bits: 8. Parity: None.

5.2 Person Detection Pipeline

Per processing frame:

1. Receive R, G, B matrices (120x160) from V4L2 Source block.
2. Assemble uint8 RGB frame: `frame(:, :, 1) = R'; frame(:, :, 2) = G'; frame(:, :, 3) = B';`
3. Resize to 320x320 using `imresize` (required by Tiny YOLOv4 input specification).
4. Run `detect(yolov4Obj, frame320, 'Threshold', 0.4)` — returns `bboxes`, `scores`, `class labels`.
5. Filter detections to retain only the 'person' class.
6. Pass filtered bounding boxes to `lockAndDecide()` for tracking and command generation.

5.3 Frame-Skip Optimisation

YOLOv4 inference (~700–800ms per call) is too slow to run every frame. A persistent frame counter introduces selective skipping:

- YOLOv4 runs only on every 3rd frame (frames 1, 4, 7, ...).

- On skipped frames, the last known bounding boxes are reused.
- The serial command (heartbeat byte) is still sent every frame — essential for watchdog safety.

This reduces YOLO calls to $\sim 33\%$ of frames while maintaining continuous motor control and watchdog heartbeat, bringing end-to-end latency to approximately 1 second.

5.4 Tracking and Lock Logic (lockAndDecide)

A persistent target lock prevents the robot from reacting to irrelevant detections and handles brief occlusions.

Lock Acquisition

When no target is locked and a new detection is available, the system selects the person bounding box whose centre is closest to the frame centre (160, 160 px). This biases tracking toward the most prominent (closest) person.

Lock Maintenance via IoU

On subsequent detection frames, the locked box is updated by finding the detected box with the highest Intersection over Union (IoU) with the current locked box. $\text{IoU} = \text{area of overlap} / \text{area of union}$. If IoU exceeds 0.25 (relaxed from typical 0.5 to tolerate position shifts during skipped frames), the lock is updated and the miss counter resets.

Lock Drop

After 6 consecutive detection frames with no matching box ($\text{MISS_TOLERANCE} = 6$, approximately 5 seconds), the lock is dropped and the robot stops.

5.5 Motor Command Decision

Condition	Command
Box area $> 15,000 \text{ px}^2$ (too close)	S — Stop
Person centre $X > \text{frame centre} + 46 \text{ px}$	R — Right turn
Person centre $X < \text{frame centre} - 46 \text{ px}$	L — Left turn
Person centred within deadband	F — Forward
No lock	S — Stop

5.6 Communication Protocol

The Pi-to-robot communication uses a simple, robust single-byte protocol:

- Protocol: single ASCII character per Simulink timestep — ‘F’, ‘L’, ‘R’, ‘S’.
- Transport: UART over USB (ATmega2560 UART2, PH0/PH1 pins). 115200 baud, 8-N-1.
- Frequency: one byte sent every Simulink step ($\sim 100\text{--}300\text{ms}$).
- Heartbeat: byte sent every frame regardless of command change. This keeps the ATmega watchdog alive.

5.7 Watchdog Safety Mechanism

A hardware watchdog timer on the ATmega2560 provides a critical safety layer:

- WDT timeout: 1 second (WDTO_1S). Set to 1s rather than 500ms to accommodate YOLO inference spikes of 700–800ms.
- `wdt_reset()` called inside the UART2 RX ISR on every received byte.
- If the Pi freezes (OS crash, Simulink hang, USB disconnect), bytes stop arriving. After 1 second, WDT fires, MCU resets, `main()` immediately calls `motor_stop()`.
- MCUSR cleared before re-enabling WDT to prevent a reset loop.

5.8 MATLAB Source Code

5.8.1 detectPersons — Top-Level Entry Point

```

1 function serialCmd = detectPersons(R, G, B)
2 %#codegen
3 serialCmd = uint8(83); % Default 'S' (Stop)
4
5 persistent frameCount;
6 if isempty(frameCount), frameCount = int32(0); end
7 frameCount = frameCount + int32(1);
8
9 DETECT_EVERY = int32(3);
10 isDetectionFrame = (mod(frameCount, DETECT_EVERY) == int32(1));
11
12 frame = zeros(120, 160, 3, 'uint8');
13 frame(:, :, 1) = uint8(R)';
14 frame(:, :, 2) = uint8(G)';
15 frame(:, :, 3) = uint8(B)';
16 frame320 = imresize(frame, [320 320]);
17
18 persistent lastPersonBoxes;
19 if isempty(lastPersonBoxes), lastPersonBoxes = zeros(10, 4); end
20
21 if isDetectionFrame
22     personBoxes = yoloDetect(frame320);
23     lastPersonBoxes = personBoxes;
24 else
25     personBoxes = lastPersonBoxes;
26 end
27
28 cmd = lockAndDecide(personBoxes, isDetectionFrame);
29 serialCmd = uint8(cmd);

```

5.8.2 yoloDetect — Model Inference

```

1 function personBoxes = yoloDetect(frame)
2 %#codegen
3 persistent yolov4Obj;
4 if isempty(yolov4Obj)
5     yolov4Obj = coder.loadDeepLearningNetwork('tinyyolov4coco.mat');
6 end

```

```
7
8 [bboxes, ~, labels] = detect(yolov4Obj, frame, 'Threshold', 0.4);
9
10 personBoxes = zeros(10, 4);
11 if ~isempty(bboxes)
12     labelStr = cellstr(labels);
13     personIdx = strcmp(labelStr, 'person');
14     pb = bboxes(personIdx, :);
15     if ~isempty(pb)
16         n = min(size(pb,1), 10);
17         personBoxes(1:n, :) = pb(1:n, :);
18     end
19 end
```

5.8.3 ATmega2560 UART2 ISR

```
1 ISR(USART2_RX_vect) {
2     uint8_t byte = UDR2;
3     switch (byte) {
4         case 'F': motor_forward(); break;
5         case 'L': motor_left(); break;
6         case 'R': motor_right(); break;
7         case 'S': motor_stop(); break;
8         default: motor_stop(); break;
9     }
10    wdt_reset();
11 }
```

Chapter 6

Phase 1 Testing and Results

6.1 Test Cases

Test	Input Condition	Expected	Result
Camera lag — 32-bit OS	Webcam on Pi, 32-bit Bookworm	< 2s lag	~10s lag — FAILED
Camera lag — 64-bit OS	Webcam on Pi, 64-bit Bookworm	< 2s lag	~8s lag — Partial
Camera lag — no display block	Removed To Video Display	< 2s lag	~7s lag — Partial
Camera lag — frames skipped	Detection every third frame	< 2s lag	~1s lag — PASS
Person detection accuracy	Single person in frame	Bounding box drawn	Detected correctly — PASS
Left turn trigger	Person moves to right of frame	Robot turns right	Working — PASS
Right turn trigger	Person moves to left of frame	Robot turns left	Working — PASS
Forward motion	Person centred in frame	Robot moves forward	Working — PASS
Stop — too close	Person very close to camera	Robot stops	Working — PASS
Watchdog safety	Simulink killed mid-run	Robot stops in 1s	Working — PASS
Detection skip (no person)	No person in frame	Robot stops	Working — PASS

6.2 Performance Observations

- End-to-end latency (frame capture to motor response): approximately 1 second.
- Primary latency contributor: Tiny YOLOv4 inference on Pi 5 (~700–800ms on inference frames, ~100ms on skipped frames).
- Frame-skip optimisation provided the most significant latency improvement.
- OS upgrade from 32-bit to 64-bit Bookworm reduced lag from ~10s to ~8s. Removal of ‘To Video Display’ block reduced further.

- Detection threshold of 0.4 balanced false positives vs missed detections at distance.
- Tracking remains stable during brief occlusion due to 6-frame miss tolerance.

Chapter 7

Challenges and Solutions (Complete)

Challenge	Root Cause	Solution / Outcome
10-second camera lag	32-bit OS bottleneck on Pi 5's 64-bit hardware	Switched to 64-bit Raspberry Pi OS Bookworm
Persistent lag after OS switch	Simulink 'To Video Display' block rendering overhead	Removed display block from final model
High latency from YOLO on every frame	Tiny YOLOv4 inference ~700–800ms per call	Frame-skip: run YOLO every 3rd frame only
Robot not stopping on Pi freeze	No watchdog in initial firmware	Implemented ATmega2560 WDT (WDTO_1S) with <code>wdt_reset()</code> in UART ISR
WDT firing during YOLO spikes	Initial WDT timeout was 500ms; YOLO spikes hit 700ms	Increased WDT timeout to 1 second
Jittery motor commands	Command sent only on detection — gaps caused spurious stops	Send heartbeat byte every frame, not only on detection
Codegen compatibility	Variable-size arrays not supported in Simulink codegen	Fixed output array sizes (<code>zeros(10, 4)</code> for bounding boxes)
YOLOv8n — same lag as YOLOv4	Synchronous Simulink model blocks on ARM inference regardless of model choice	Confirmed architectural limitation; began exploring alternatives
Haar cascade — no detections	Low resolution, off-angle faces, cluttered background	Abandoned; confirmed cascade requires controlled frontal face input
MobileNet-SSD — no bounding boxes	MATLAB toolbox provides classifier only, not detector	Abandoned; not applicable to tracking requirement
HOG detector — false positives everywhere	Lab furniture matches pedestrian HOG feature patterns	Abandoned; unsafe robot behaviour
ACF detector — jittery tracking	Bounding box instability inherent to boosted cascade at video frame rate; Kalman and EMA smoothing only partially mitigated jitter	Abandoned; demonstrated real-time camera is possible classically, but tracking robustness insufficient

Challenge	Root Cause	Solution / Outcome
YOLOX — same lag	Simulink synchronous step is the bottleneck regardless of model	Confirmed; decision made to move to Python
ROS study — not adopted	Excessive complexity for standalone robot; Python threading sufficient	Python implementation chosen instead

Chapter 8

Key Technical Learnings

- OS architecture matters significantly: 64-bit vs 32-bit on Pi 5 has measurable real-world impact beyond benchmarks, particularly for V4L2 frame handling on 64-bit Cortex-A76 cores.
- Simulink's synchronous execution model is fundamentally incompatible with variable-latency deep learning inference on an ARM CPU without hardware acceleration. The model step blocks until inference completes, stalling the entire pipeline.
- In Simulink codegen workflows, all arrays must be fixed-size. Dynamic allocation is not supported and must be worked around by pre-allocating maximum-size arrays.
- A heartbeat-based watchdog design is more robust than event-driven communication: sending a byte every frame ensures the safety mechanism is continuously validated.
- Frame-skipping is a simple but highly effective technique for deploying heavy inference models on resource-constrained hardware without sacrificing control loop continuity.
- IoU-based target re-identification is far more robust than nearest-bounding-box matching for maintaining tracking continuity across frames.
- Model selection alone cannot solve architectural bottlenecks: YOLOv4, YOLOv8n, and YOLOX all produced similar latency within Simulink because the constraint was execution architecture, not model size.
- Classical detectors (Haar, HOG, ACF) are fast on CPU but have brittle accuracy requirements. ACF uniquely demonstrated real-time camera throughput within Simulink — confirming that the latency bottleneck is deep learning inference, not Simulink itself — but its tracking jitter made it unsuitable for reliable robot control.
- Python's threading and process model provides a simpler, more direct solution to the synchronous/asynchronous problem than ROS for single-robot applications.

Chapter 9

Phase 2: Python Implementation (Completed)

9.1 Overview and Final Architecture

The Python phase of the project replaced the MATLAB/Simulink pipeline entirely. The complete system now runs as a single Python application on the Raspberry Pi 5, structured as three concurrent threads coordinated through shared, lock-protected state: a camera capture thread, a voice command listener thread, and a main tracking/control thread, alongside a Flask web server running on the main process for live monitoring. This multi-threaded design directly solves the synchronous blocking problem that limited the Simulink implementation — camera capture, speech recognition, and motor command transmission all proceed independently of however long YOLO inference takes on a given frame.

At a high level, the final pipeline is:

```
ThreadedCamera (background thread)
  cv2.VideoCapture -> continuously refreshed shared frame buffer

Voice Listener (background thread)
  Vosk offline ASR -> robot_active flag (START / STOP)

Main Tracking Loop (primary thread)
  latest frame -> YOLOv8n (face or person) -> FaceLock -> decide_command()
  -> RobotSerial.send() (debounced) -> ATmega2560 over UART
  -> annotated frame -> shared buffer for MJPEG streaming

Flask App (primary thread, app.run())
  /video_feed -> live MJPEG stream of annotated frame
  /           -> simple dashboard page
```

The ATmega2560 firmware was also rewritten for this phase. It still receives single-character commands from the Pi over UART2, but it no longer blindly executes them: a front-facing Sharp GP2D12 IR distance sensor and a side-facing IR proximity sensor give the microcontroller its own local obstacle-avoidance reflex that can temporarily override the Pi's tracking command, drive an avoidance manoeuvre, and then resume obeying the Pi. This is described in detail in [Chapter 10](#).

9.2 Detection Pipeline

Detection uses Ultralytics YOLOv8n, loaded once at start-up. The application prefers a face-specific checkpoint (`yolov8n-face-lindevs.pt`) when present on disk, and falls back to the standard

COCO-trained `yolov8n.pt` (filtered to class 0, person) when it is not. This dual-model approach means the same codebase can run either as a face tracker or a person tracker without code changes, depending on which weights file is available on the deployed Pi.

- **Inference resolution:** frames are processed at `imgsz=320` rather than the model's native 640, trading a small amount of accuracy for materially faster inference on the Pi 5 CPU.
- **Confidence threshold:** `CONF_THRESHOLD = 0.35`, tuned to reject the spurious low-confidence boxes seen during the MATLAB phase while still catching faces at moderate distance.
- Each detection is reduced to a 6-tuple `(x1, y1, x2, y2, confidence, class_id)` before being handed to the lock logic, keeping the tracking code independent of which underlying model produced the box.

9.3 Threaded Camera Capture

The `ThreadedCamera` class wraps `cv2.VideoCapture` in a dedicated background thread that continuously calls `cap.read()` and stores the most recent frame behind a `threading.Lock`. The main loop simply calls `cam.read()` to fetch whatever frame is currently available, rather than blocking on the camera driver. This decouples camera frame rate from YOLO inference time and was a direct fix for the camera-frame-lag problem that dominated the MATLAB/Simulink phase — the camera buffer is never allowed to back up waiting for a slow consumer.

- Capture resolution fixed at 320x240 (`FRAME_W x FRAME_H`) to match the YOLO inference size and minimise resize overhead per frame.
- A 10ms sleep between reads inside the capture thread avoids needlessly pegging a CPU core while still keeping the buffer fresh well above the control loop's required rate.
- `read()` returns a defensive `.copy()` of the frame so the main thread never reads a frame that is being concurrently overwritten by the capture thread.

9.4 Target Lock and Proportional Steering

9.4.1 FaceLock: Centre-Point Target Persistence

The `FaceLock` class is a direct Python port of the lock-and-decide concept used in the MATLAB phase, adapted to run per-frame in real time. Rather than IoU matching (used in the MATLAB phase), the Python implementation matches the previously locked box to the new frame's detections by nearest centre-point distance, which is computationally cheaper and behaves equivalently for the single-target tracking scenario used here.

- **Lock acquisition:** when no target is currently locked, the detection with the highest confidence score is selected as the new lock.
- **Lock maintenance:** on each subsequent frame, the detection whose box centre is closest (Euclidean distance) to the previously locked box's centre is selected as the updated lock.
- **Grace period on loss:** if no detections are present in a frame, the lock is not dropped immediately. A `lost_since` timestamp starts a countdown of `LOCK_LOST_SEC = 5.0` seconds; only if no detection reappears within that window is the lock cleared. This prevents the robot from giving up on a brief detection miss (motion blur, momentary occlusion, a head turn).

9.4.2 Proportional Steering: `decide_command()`

Where the MATLAB phase used a single binary Left/Right decision, the Python phase implements a three-band proportional controller around the horizontal pixel error between the locked target's centre and the frame centre:

Condition ($ \text{error} $ in px)	Command	Meaning
$ \text{error} \leq \text{CENTER_TOL} (75)$	F	Target centred — drive straight forward
$75 < \text{error} \leq 115$ ($\text{CENTER_TOL} + \text{PROP_THRESHOLD}$)	$1/r$	Small offset — soft turn at reduced motor speed
$ \text{error} > 115$	L / R	Large offset — hard turn at full turning speed

This graduated response reduces the oscillation ("wobble") that a binary controller produces when the target sits near the deadband edge, since small corrections are now driven at reduced speed instead of triggering the same full-power turn as a large correction.

9.4.3 Momentum on Brief Loss and Serial Debounce

Two further stability measures were added at the Python layer, independent of the ATmega side:

- **Momentum buffer:** if the lock is briefly lost, the last good command (`momentum_cmd`) is repeated for up to 60 frames (roughly two seconds at the achieved frame rate) before the robot is commanded to stop, smoothing over single-frame detection misses without needing a full re-acquisition.
- **RobotSerial debounce:** outgoing serial writes require `DEBOUNCE_N = 3` consecutive identical decisions and a minimum `CMD_INTERVAL = 0.08s` between writes before a new command byte is actually transmitted to the ATmega2560. This filters frame-to-frame command flicker at the source, on top of the anti-spin logic implemented in firmware (Section 10.4).

9.5 Voice Command Subsystem (Start / Stop Control)

A fully offline voice control layer was added so the robot can be armed or halted by spoken command without any network dependency — important given the robot operates untethered from any host PC.

- **Speech engine:** Vosk, a lightweight offline speech recognition toolkit, paired with `sounddevice` for microphone capture at 16 kHz.
- **Restricted grammar:** rather than general-purpose transcription, the recognizer is constrained to the vocabulary `["start", "stop", "[unk]"]`. Limiting the grammar to exactly the words needed both reduces CPU load and substantially cuts misrecognition, since the model only has to distinguish between two keywords and a generic "unknown" bucket.
- **Partial-result polling:** the listener reads `Vosk's PartialResult()` during ongoing speech rather than waiting only for `AcceptWaveform()` to finalise an utterance. This removes the natural silence delay before a keyword is acted on, so "START" takes effect essentially as soon as it is spoken rather than after the speaker pauses.
- **Global state flag:** recognised keywords toggle a shared `robot_active` boolean. While `robot_active` is `False`, the main loop unconditionally sends the stop command 'S' to the ATmega2560 regard-

less of what the tracker has decided — voice STOP is a hard override on top of the tracking logic, not merely a suggestion to it.

- **Immediate halt on STOP:** in addition to setting the flag, the voice thread also calls `robot_instance.send("stop")` directly the moment "stop" is recognised, rather than waiting for the next iteration of the main loop to notice the flag change.

This subsystem runs in its own daemon thread (`voice_listener_thread`) so that microphone polling and speech inference never block the camera/tracking loop or the Flask server.

9.6 Live Video Streaming Dashboard

A small Flask application exposes the robot's live annotated camera feed over the local network as an MJPEG stream, allowing the system to be monitored from a phone or laptop browser without attaching a monitor to the Pi.

- `/video_feed` serves a `multipart/x-mixed-replace` MJPEG stream generated from the most recently annotated frame, JPEG-encoded at quality 50 to keep bandwidth and encode time low on the Pi's CPU.
- `/` serves a minimal HTML dashboard page embedding the video stream.
- Each streamed frame is annotated with: green boxes for every raw detection, a red box for the currently locked target, vertical guide lines marking the centre and the proportional-control deadband, and text overlays showing the live voice ACTIVE/HALTED state and the command currently being sent.
- The Flask server (`app.run(host="0.0.0.0", port=5000, threaded=True)`) runs on the main thread while the entire tracking pipeline runs in a background daemon thread, so the dashboard remains responsive even while inference is in progress.

Chapter 10

Obstacle Avoidance Subsystem (ATmega2560 Firmware)

10.1 Motivation and Control Philosophy

While the Raspberry Pi decides where the robot should go (track a face/person and turn toward it), it has no way of knowing what is physically in front of the robot at a given instant beyond what the camera sees — and the camera’s field of view does not reliably cover close-range obstacles below or to the side of frame. Rather than attempting to fold obstacle detection into the vision pipeline, obstacle avoidance was implemented as an independent safety/manoeuvring reflex inside the ATmega2560 firmware itself, sitting logically beneath the Pi’s high-level tracking commands.

This produces a simple two-layer (subsumption-style) control architecture: the Pi continuously streams a high-level intent command (‘F’, ‘L’, ‘R’, ‘I’, ‘r’, or ‘S’), and the ATmega2560 normally just executes it — but if its own front-facing distance sensor detects an obstacle inside a safe threshold while the Pi is commanding any kind of motion, the firmware suspends obedience to the Pi, runs a fixed avoidance manoeuvre autonomously, and only returns control to the Pi’s commands once the manoeuvre completes. The Pi is not aware the manoeuvre happened; it simply continues sending tracking commands throughout, and the robot resumes following them afterward.

10.2 Sensors Used

Sensor	ADC Channel	Role
Sharp GP2D12 IR distance sensor	Channel 11 (SHARP_FRONT_CH)	Front-facing analog distance sensor. Converted from raw ADC counts to centimetres via the manufacturer’s inverse power-law response curve.
IR proximity sensor (right side)	Channel 7 (PROX_CH_4)	Side-facing proximity sensor used to detect when the robot has cleared the side of an obstacle during the wall-following step of the avoidance manoeuvre.

The front sensor’s raw 8-bit ADC reading is converted to an estimated distance using the standard GP2D12 characteristic curve,

$$\text{dist_mm} = 10 \times 2799.6 \times \text{reading}^{-1.1546},$$

clamped to a maximum of 800mm to avoid the curve's instability at very low reflected signal. The result is divided by 10 in the main loop to obtain centimetres. SHARP_STOP_CM is set to 15cm — the trigger distance at which the avoidance manoeuvre begins.

10.3 Avoidance State Machine

Obstacle avoidance is implemented as an explicit finite state machine (`obst_state`), evaluated every iteration of the 1ms main loop whenever the Pi is commanding any motion other than stop. The five states execute strictly in sequence and cannot be interrupted by a new Pi command until the sequence returns to idle:

State	Action	Exit Condition
ST_REVERSE	Drive in reverse at full speed to create clearance from the obstacle.	AVOID_REV_MS elapsed (400ms)
ST_TURN_LEFT	Pivot left at reduced (turning) speed, roughly a 90° turn.	AVOID_TURN90_MS elapsed (450ms)
ST_WALL_FOLLOW	Drive forward past the obstacle, continuously sampling the right-side IR sensor.	Right-side reading exceeds RIGHT_WALL_THRES (100) — i.e. the obstacle's side has been passed and open space is detected
ST_SAFETY_FWD	Continue forward briefly to ensure the rear wheels have also cleared the obstacle, not just the front sensor.	AVOID_SAFETY_MS elapsed (600ms)
ST_TURN_RIGHT	Pivot right roughly 90° to restore the robot's original heading.	AVOID_TURN90_MS elapsed (450ms) — state returns to ST_IDLE, Pi commands resume normal control

In short, the manoeuvre is: back away, turn left, drive past the obstacle while watching the right-side sensor for the moment the obstacle's far edge is passed, push forward a little further for safety margin, then turn back right to resume the original heading. Once ST_IDLE is reached again, the robot immediately resumes responding to whatever command the Pi is currently sending — including re-triggering the avoidance sequence again if another obstacle is encountered.

A representative excerpt of the state machine from the main control loop:

```

1  switch (obst_state) {
2      case ST_REVERSE:
3          set_full_speed(); motor_reverse();
4          if (g_ms - obst_t0 >= AVOID_REV_MS) {
5              motor_stop(); obst_state = ST_TURN_LEFT; obst_t0 = g_ms;
6          }
7          break;
8
9      case ST_WALL_FOLLOW:
10         set_full_speed(); motor_forward();
11         if (right_prox > RIGHT_WALL_THRES) {
12             motor_stop(); obst_state = ST_SAFETY_FWD; obst_t0 = g_ms;
13         }
14         break;
15
16         /* ... ST_TURN_LEFT, ST_SAFETY_FWD, ST_TURN_RIGHT follow the same

```

```

17         timed-transition pattern, each returning to ST_IDLE in sequence
        ... */
18     }

```

10.4 Anti-Spin Turn-Burst Logic (Tracking Turns)

A second, separate stability mechanism governs ordinary left/right tracking turns (as opposed to the obstacle-avoidance turns above). Earlier in the project, sustained turn commands from the Pi caused the robot to spin continuously without re-checking whether the target had already recentred, leading to overshoot. The firmware now bursts every tracking turn into a fixed `TURN_BURST_MS = 300ms` pulse at reduced turning speed, followed by a mandatory `PAUSE_MS = 150ms` full stop, before allowing another burst. Each pause gives the Pi's next frame-based decision a chance to take effect before the robot turns further, preventing the robot from overshooting a centred target purely because several stale turn commands arrived in a row.

- Soft turns ('l' / 'r') and hard turns ('L' / 'R') both pass through the same burst/pause cycle; only the underlying motor command (`motor_left/motor_right` at `set_turn_speed`) differs based on which character was received.
- The obstacle-avoidance state machine (Section 11.3) is independent of this burst/pause logic — avoidance turns run at their own fixed durations and are not subject to the anti-spin pausing, since they are time-critical safety manoeuvres rather than visually-corrected tracking adjustments.

10.5 LCD Status Display

A 16x2 character LCD, driven in 4-bit mode through PORTC, gives a live readout of robot state for on-bench debugging without needing a serial monitor:

- Line 1: current front distance in cm, current right-side proximity reading, and the raw command character last committed from the Pi (e.g. `F:12 R:145 C:F`).
- Line 2: a human-readable status string that switches automatically between `HALTED`, one of the five `AVOID...` obstacle-avoidance stage labels, `ANTI SPIN PAUSE`, or `TRACKING FWD / LEFT / RIGHT / SOFT L / SOFT R` depending on the current state.
- The display refreshes roughly every 200 main-loop iterations (`lcd_div` counter) rather than every loop, since LCD writes are comparatively slow and do not need to update at millisecond resolution.

10.6 Watchdog Safety (Retained from Phase 1)

The hardware watchdog timer mechanism carried over unchanged from the MATLAB phase: `WDTO_1S` is enabled at start-up, and `wdt_reset()` is called from inside the `USART2_RX_vect` interrupt service routine whenever a recognised command byte ('F', 'L', 'R', 'l', 'r', 'S') is received, as well as once per main-loop iteration. If the Raspberry Pi stops sending bytes for any reason — process crash, USB disconnect, OS hang — the watchdog fires within one second, the ATmega2560 resets, and `main()` immediately calls `motor_stop()` before re-entering its normal initialisation. This remains the system's last line of defence and operates completely independently of both the tracking logic and the new obstacle-avoidance state machine.

Chapter 11

Phase 2 Testing and Results

11.1 Test Cases

Test	Input Condition	Expected	Result	
Face/person tracking, centred	Target stands centred in frame	Robot drives forward (F)	Working PASS	—
Proportional soft turn	Target offset 75–115px from centre	Robot issues soft turn (l/r) at reduced speed	Working PASS	—
Proportional hard turn	Target offset > 115px from centre	Robot issues hard turn (L/R) at full turning speed	Working PASS	—
Brief detection loss	Target momentarily occluded (< 2s)	Robot continues last command (momentum), does not stop	Working PASS	—
Extended detection loss	Target leaves frame for > 5s	Lock drops, robot stops	Working PASS	—
Voice START	Spoken keyword "start"	<code>robot_active = True</code> , tracking commands take effect	Working PASS	—
Voice STOP	Spoken keyword "stop"	Immediate 'S' sent, <code>robot_active = False</code>	Working PASS	—
Voice override during tracking	"stop" spoken while actively following a centred target	Robot halts despite a valid tracking lock	Working PASS	—
Obstacle avoidance trigger	Object placed within 15cm of front sensor while Pi commands F	Robot reverses, turns left, bypasses object, turns right, resumes tracking	Working PASS	—
Obstacle avoidance during turn command	Object detected at 15cm while Pi commands L/R	Avoidance sequence still triggers and completes before resuming turn	Working PASS	—
Anti-spin pause	Sustained turn command from Pi over multiple frames	Robot bursts turn for 300ms, pauses 150ms, re-evaluates	Working PASS; overshoot on centred target reduced	—

Test	Input Condition	Expected	Result	
Serial debounce	Tracking decision flickers between F and l across consecutive frames	No command byte sent until 3 consecutive identical decisions	Working PASS	—
Watchdog safety	Python process killed mid-run	ATmega2560 resets and stops motors within 1s	Working PASS	—
MJPEG dashboard	Browser connects to Pi IP on port 5000	Live annotated video stream with status overlay	Working PASS	—

11.2 Performance Observations

- Removing the synchronous Simulink execution model eliminated the multi-second end-to-end lag observed in Phase 1; the threaded camera and asynchronous control loop keep frame staleness to a single-digit number of milliseconds regardless of YOLO inference time.
- Running inference at `imgsz=320` instead of the model default (640) was the single largest contributor to usable frame rate on the Pi 5 CPU, at an acceptable cost to detection range/accuracy for the lab-scale tracking distances used.
- The dual-band proportional steering (soft vs. hard turn) visibly reduced the "hunting" behaviour around the centre deadband that was present with a purely binary Left/Right decision.
- The anti-spin turn-burst/pause cycle measurably reduced overshoot when the robot reaches a centred target after a string of turn commands, at the cost of a slightly less fluid-looking turning motion.
- The Vosk grammar restriction to `["start", "stop", "[unk]"]` kept voice recognition responsive enough for near-real-time START/STOP control on the Pi's CPU, which would not have been practical with a full, unconstrained vocabulary model at this compute budget.
- The obstacle-avoidance manoeuvre's fixed timings (reverse 400ms, turn 450ms, safety push 600ms) were empirically tuned on the bench for the Firebird V's specific motor speed and chassis size; they are not adaptive to obstacle size or approach angle, which is noted as a future improvement in Chapter 14.

Chapter 12

Challenges and Solutions (Phase 2)

Challenge	Root Cause	Solution / Outcome
Camera frame lag persisted naively with synchronous capture	<code>cv2.VideoCapture.read()</code> blocks the calling thread on slow frames	Introduced <code>ThreadedCamera</code> : dedicated capture thread continuously refreshes a shared frame buffer; main loop never blocks on the camera
Jittery, oscillating motor commands near the centre deadband	Binary Left/Right decision treated small and large offsets identically	Added a proportional soft-turn band ('l'/'r' at reduced speed) between the deadband and the full-turn threshold
Robot overshooting a centred target after a run of turn commands	Sustained turning with no pause to re-evaluate the latest frame	Implemented firmware-side anti-spin turn-burst (300ms burst, 150ms pause) so each turn segment is re-validated against fresh tracking data
Spurious frame-to-frame command flicker reaching the motors	Per-frame detection noise occasionally flips the decided command for a single frame	Added serial-side debounce in <code>RobotSerial.send()</code> requiring 3 consecutive identical decisions and a minimum 80ms interval before transmitting
Robot stopping abruptly on momentary detection misses	Lock dropped immediately whenever a single frame had no detections	Added a momentum buffer (continue last good command for up to ~2s) and a 5-second <code>FaceLock</code> grace period before dropping the lock
No way to safely halt the robot without physical access once untethered	All control previously came only from the vision pipeline's own decisions	Added an offline Vosk-based voice command thread for hands-free START/STOP, acting as a hard override on the main loop
Voice recognition unreliable / slow with a general vocabulary model	Full-vocabulary ASR is both slower and more error-prone on Pi 5 CPU for two simple keywords	Constrained the Vosk grammar to <code>["start", "stop", "[unk]"]</code> and polled partial results to reduce both latency and misrecognition

Challenge	Root Cause	Solution / Outcome
Robot had no way to react to obstacles outside the camera's effective tracking range	Vision pipeline tracks a person/face, not general obstacles, and is not reliable at very close range	Implemented an independent ATmega2560-side obstacle-avoidance state machine driven by a front Sharp GP2D12 sensor, layered beneath the Pi's tracking commands
Risk of the avoidance manoeuvre clearing the front of an obstacle but clipping it with the rear wheels	Front sensor only confirms the front of the robot has cleared	Added a dedicated <code>ST_SAFETY_FWD</code> state: an extra timed forward push after the side-sensor clears, before turning back to heading
Need to resume the correct heading after an avoidance manoeuvre without Pi awareness of the detour	Pi has no knowledge that an avoidance sequence occurred	Designed the manoeuvre to be a closed loop returning to the same approximate heading (reverse $\rightarrow$ left $90^\circ \rightarrow$ forward $\rightarrow$ right 90°), so resuming Pi commands afterward behaves as if no detour happened

Chapter 13

Future Scope

13.1 Planned Hardware Improvements

- **Battery integration:** replace the bench charger with a 12V LiPo battery and buck converter (to 5V/3A for the Pi 5), mounted on the Firebird V chassis for fully untethered operation.
- **Ultrasonic or ToF distance sensing:** complement or replace the analog Sharp GP2D12 with an HC-SR04 or VL53L1X time-of-flight sensor for more consistent distance measurement across surfaces, reducing the IR sensor's sensitivity to target reflectivity and ambient IR.
- **Pi Camera Module:** replace the USB webcam with a Pi Camera Module 3 for lower-latency V4L2 access and better low-light performance.
- **Side sensors on both flanks:** add a left-side IR sensor mirroring the existing right-side one, so the avoidance manoeuvre can dynamically choose which side to bypass an obstacle on, rather than always turning left first.

13.2 Planned Software Improvements

- **Model export and quantisation:** export the YOLOv8n weights to ONNX or INT8 TFLite format to further reduce per-frame inference latency on the Pi 5's ARM cores.
- **Adaptive obstacle-avoidance timing:** replace the fixed-duration avoidance states with distance- and speed-aware timing, so the manoeuvre scales correctly with obstacle size and approach speed rather than using one fixed set of durations.
- **Expanded voice vocabulary:** extend the Vosk grammar beyond START/STOP to support simple directional overrides (e.g. "left", "right", "come here") for manual assistance during testing.
- **Face recognition:** extend beyond generic face/person detection to identify and lock onto a specific registered individual using a lightweight embedding model.
- **Kalman-filtered box prediction:** predict the locked target's position between inference cycles to further smooth steering decisions under variable frame rate.
- **Outdoor and variable-lighting validation:** test and re-tune detection confidence and proportional-control thresholds under outdoor lighting, where the controlled lab environment's assumptions may not hold.

Chapter 14

Conclusion

This project documents a complete engineering journey — from an initial MATLAB/Simulink prototype, through systematic exploration of six alternative detection approaches and a study of ROS, to a fully working, Python-based, voice-controllable, obstacle-aware face/person-tracking robot — over the course of an approximately six-week internship at KPIT Apex Lab.

Phase 1 demonstrated a working person-tracking robot entirely within MATLAB and Simulink using Tiny YOLOv4, but was fundamentally limited by the synchronous execution model's incompatibility with variable-latency deep learning inference on an ARM CPU without hardware acceleration. Systematic evaluation of YOLOv8n, Haar cascades, MobileNet-SSD, HOG, a custom ACF detector, YOLOX, and ROS confirmed that the limitation was architectural rather than tied to any specific model choice.

Phase 2 resolved this by moving the entire pipeline to Python on the Raspberry Pi 5: a threaded camera capture loop decoupled frame acquisition from inference, Ultralytics YOLOv8n provided the detection backbone, and a centre-point FaceLock with proportional (soft/hard) steering produced smoother tracking behaviour than the earlier binary controller. Two further capabilities were layered on top of this working tracker during the final stretch of the internship: an offline, Vosk-based voice command subsystem giving hands-free START/STOP control with a hard override on the tracking logic, and an independent ATmega2560-side obstacle-avoidance state machine that allows the robot to autonomously detect, bypass, and recover heading around obstacles encountered while following its target — all without requiring the Pi to be aware that an avoidance manoeuvre took place.

The result is a robot that tracks a person or face in real time, can be safely armed or halted by voice, and reacts to physical obstacles in its path independently of the vision pipeline — a layered control architecture combining a high-level perception/decision loop on the Pi with a low-level safety/manoeuvring reflex on the microcontroller. The technical problem-solving carried out across both phases — diagnosing architectural bottlenecks, comparing classical and deep-learning detectors, designing debounced and proportional control loops, implementing offline speech recognition under tight compute constraints, and building a finite-state obstacle-avoidance reflex on bare-metal firmware — represents broadly applicable learning for real-time embedded robotics and computer vision system development.

References

1. MathWorks. *MATLAB and Simulink R2026a Documentation*. <https://www.mathworks.com/help/>
2. MathWorks. *Tiny YOLOv4 — Deep Learning Toolbox Model*. <https://www.mathworks.com/help/deeplearning/>
3. MathWorks. *Raspberry Pi Blockset — V4L2 Video Source*. <https://www.mathworks.com/help/supportpkg/raspberrypi/>
4. Ultralytics. *YOLOv8 Documentation*. <https://docs.ultralytics.com/>
5. eYantra, IIT Bombay. *Firebird V ATmega2560 Hardware Manual*.
6. Atmel Corporation. *ATmega2560 Datasheet*. <https://www.microchip.com/>
7. Sharp Corporation. *GP2D12 IR Distance Sensor Datasheet*.
8. Viola, P., Jones, M. *Rapid Object Detection using a Boosted Cascade of Simple Features*. CVPR 2001.
9. Dalal, N., Triggs, B. *Histograms of Oriented Gradients for Human Detection*. CVPR 2005.
10. Bochkovskiy, A., Wang, C., Liao, H. *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv:2004.10934, 2020.
11. Ge, Z. et al. *YOLOX: Exceeding YOLO Series in 2021*. arXiv:2107.08430, 2021.
12. Alpha Cephei. *Vosk Offline Speech Recognition Toolkit*. <https://alphacephei.com/vosk/>
13. OpenCV. *OpenCV-Python Documentation*. <https://docs.opencv.org/>
14. Pallets Projects. *Flask Documentation*. <https://flask.palletsprojects.com/>
15. Open Robotics. *ROS 2 Documentation*. <https://docs.ros.org/>
16. MathWorks. *MATLAB Onramp: Machine Learning, Computer Vision, Deep Learning*. <https://matlabacademy.mathworks.com/>
17. MathWorks. *MATLAB Deep Learning Model Hub (GitHub)*. <https://github.com/matlab-deep-learning/MATLAB-Deep-Learning-Model-Hub>